Método BFGS Aplicado a Regresión Logística

Adrian, Alejandro, Miguel, Nelson

07/12/2021

Introducción

- Los métodos de Cuasi-Newton son una clase de métodos que empezaron a desarrollarse alrededor de los años 1950 por falta de poder computacional y memoria que tenían las computadoras en esa época.
- Uno de esos métodos es BFGS, llamado así por la gente que primero lo desarrolló, Broyden, Fletcher, Goldfarb y Shanno.

Introducción

- Los métodos de cuasi-Newton solamente utilizan el gradiente de la función objetivo para poder iterar y llegar a aproximaciones de los puntos óptimos locales.
- La mejora puede ser considerable, especialmente cuando los problemas tienen muchas dimensiones o las funciones a aproximar son extremadamente complejas.
- Otro beneficio para este tipo de métodos es que como no se requiere una segunda derivada, entonces son generalmente mas eficientes que los métodos de Newton.

Introducción

- Explotamos las fortalezas de este tipo de métodos generando una implementación del método BFGS para resolver un problema de estimación de máxima verosimilitud.
- Hacemos una regresión logística, y ajustamos los parámetros para poder minimizar la pérdida logarítmica del ajuste del modelo.
- Esto se hace resolviendo un problema de minimización en términos de los valores de los coeficientes del modelo.

Problema a Resolver

Los datos utilizados en este problema son referentes a derrames cerebrales (*stroke*), la segunda mayor causa de muerte a nivel mundial de acuerdo con la Organización Mundial de la Salud (OMS). La base de datos fue obtenida de Kaggle y lleva por nombre Stroke Prediction Dataset. Con el modelo desarrollado utilizamos características importantes sobre los pacientes para predecir la ocurrencia de un derrame.

Problema a Resolver

Las variables predictoras son:

- gender: genero del paciente.
- age: edad del paciente.
- hypertension: 0 si no padece o 1 si padece hipertensión.
- heart_disease: 0 si padece o 1 si padece una enfermedad cardiaca.
- ever_married: alguna vez casado (sí o no).
- residence_type: tipo de residencia rural o urbana.
- avg_glucose_level: nivel de glucosa promedio en sangre.
- bmi: índice de masa corporal.
- smoking_status: fumador, ha fumado, nunca fuma o desconocido.

Problema a Resolver

La variable a predecir es:

- stroke: 1 si el paciente tuvo un derrame o 0 si no.

La variable a predecir es categórica con dos niveles, por lo que el modelo elegido para clasificación es regresión logística.

Problema a Resolver

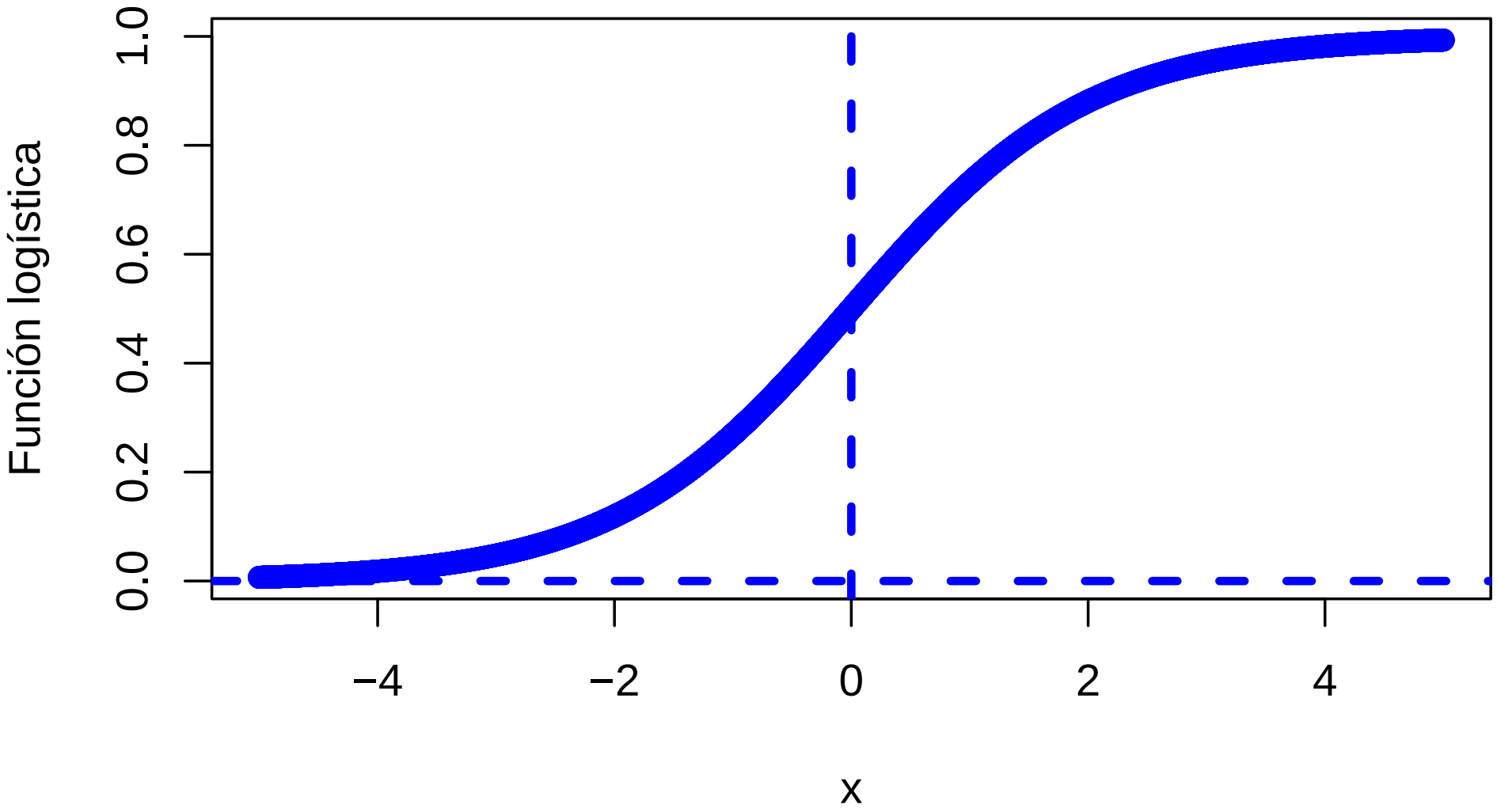
El modelo de regresión logística predice la probabilidad de que la Y pertenezca a alguna de las dos categorías posibles. Para definir un modelo p(X), partimos de un promedio ponderado de las variables predictoras:

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$$

Esta expresión es transformada por la función sigmoide o logística:

$$h(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{e^x + 1}$$

Problema a Resolver



Para valores negativos de x la función logística se acerca al valor de cero y para valores positivos de x la función toma valores cercanos a 1.

Problema a Resolver

Entonces el modelo de regresión logística queda expresado con la siguiente ecuación:

$$p(x) = h\left(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n\right)$$

La siguiente pregunta sería, ¿cómo elegimos los coeficientes de nuestro modelo?

Problema a Resolver

Dado un conjunto de datos de entrenamiento, se espera que las probabilidades de clase obtenidas en la clasificación sean lo más cercanas posible al valor de sus correspondientes categorías.

Esto es, se busca un error mínimo entre las predicciones de nuestro modelo y el valor de la variable objetivo.

Problema a Resolver

Para hacer esto, nos apoyamos de la siguiente función de costo utilizada en el problema de clasificación:

$$Cost(h(X), Y) = \begin{cases} -log(h(X)) \;\; si \;\; y = 1 \\ \\ -log(1 - h(X)) \;\; si \;\; y = 0 \end{cases}$$

Esta función puede ser reexpresada de la siguiente forma:

$$Cost(h(X), Y) = -\frac{1}{n}\left[\sum_{i=1}^{n} y_i log h(x_i) + (1 - y_i) log(1 - h(x_i))\right]$$

donde n es la cantidad de datos en el conjunto de entrenamiento.

Problema a Resolver

Esta función se generalmente es minimizada con un método iterativo ampliamente conocido como descenso en gradiente. Este método funciona bien y su implementación es relativamente sencilla, aunque para grandes conjuntos de datos llega a ser exigente computacionalmente. En este proyecto utilizamos un optimizador alternativo, conocido como BFGS en honor a sus autores. Este es un método Cuasi-Newton, el cual no requiere de una selección manual del tamaño de paso, tiene una convergencia más rápida que el descenso en gradiente y es más conveniente en términos computacionales.

Método BFGS

El método más popular entre la clase de cuasi-Newton es el método BFGS y sus derivados.

Este es un método de descenso, que formalmente, es un método que genera una secuencia de minimización de puntos $x^{(0)}, x^{(1)}, \ldots, x^{(n)} \in dom(f_0)$ con f_0 la función objetivo, con la característica de que $f_0(x^{(k+1)}) < f_0(x^{(k)})$ excepto para $x^{(k)}$ óptimo, y también, $f_0(x^{(k)}) \to p^*$ si $k \to \infty$.

Método BFGS

Para el iterado $x^{(k)}$, se puede formar usando el teorema de Taylor el siguiente modelo cuadrático:

$$m_k(p) = f_k + \nabla f_k^T p + \frac{1}{2} p^T \nabla^2 f_k p,$$

donde $f_k = f_0(x^{(k)})$ es la función objetivo evaluado en el iterado $x^{(k)}$, y p es la dirección de descenso.

Método BFGS

Lo que sigue entonces es hacer una aproximación a la Hessiana $\nabla^2 f_k \approx B_k$, donde $B_k \in \mathbb{R}^{n \times n}$ es una matriz simétrica positiva semiefinida que se actualiza a cada iteración. El problema es entonces:

$$m_k(p) = f_k + \nabla f_k^T p + \frac{1}{2} p^T B_k p.$$

Método BFGS

El elemento que resuelve este problema lo llamamos p_k, que es explicitamente

$$p_k = -B_k^{-1} \nabla f_k.$$

Notamos antes de continuar que esta dirección es muy parecida a la dirección de Newton, pero usamos la aproximación a la Hessiana en vez de la Hessiana en sí. Por lo tanto, la actualización del paso es entonces:

$$x^{(k+1)} = x^{(k)} + \alpha_k p_k$$

donde α_k es una longitud de paso que satisface las dos condiciones de Wolfe.

Condiciones de Wolfe

Estas condiciones quedan fuera del rango de este trabajo, pero, una explicación detallada se puede encontrar en [Nocedal, 3.6].

Método BFGS

El gran truco del método BFGS es esta matriz B_k. Para lidiar con los costos computacionales altos, se propuso en vez de calcular a cada iteración la Hessiana hacerle una actualización a esta matriz B_k. El desarrollo completo de esta actualización se puede ver en [Nocedal, 6.1], y es nombrada la actualización DFP:

$$B_{k+1} = (I - \rho_k y_k s_k^T) B_k (I - \rho_k s_k y_k^T) + \rho_k y_k y_k^T,$$

con

$$\rho_k = \frac{1}{y_k^T s_k},$$

donde $y_k = \nabla f_{k+1} - \nabla f_k$, y $s_k = x_{k+1} - x_k = \alpha_k p_k$.

Método BFGS

Tomando $H_k = B_k^{-1}$ por su utilidad en el cálculo de la dirección de decenso, entonces podemos ver que la actualización que se usa en el método es:

$$H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T,$$

y generalmente para la inicialización del método se toma $H_0 = I_{n \times n}$.

Con este desarrollo, podemos entones definir el algoritmo de BFGS completo.

Algoritmo BFGS

Algorithm 1: Método BFGS

Result: Aproximación a p^* dada por $x^{(k)}$
Input : Punto inicial $x^{(0)}$, tolerancia $\epsilon > 0$, $H_0 = I$
Output: Punto $x^{(k)}$, k que da el número de iteraciones

$k \leftarrow 0$;
while *Mientras* $||\nabla f_k|| > \epsilon$: **do**
    Calcular la dirección de descenso $p_k = -H_k \nabla f_k$;
    Actualizamos $x^{(k+1)} = x^{(k)} + \alpha_k p_k$ tal que α_k cumpla las condiciones de Wolfe;
    Definimos $s_k = x^{(k+1)} - x^{(k)}$;
    Definimos $y_k = \nabla f_{k+1} - \nabla f_k$;
    Actualizamos $H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T$;
    $k \leftarrow k + 1$;
end

Figure 1: BFGS

Algoritmo BFGS

- Notamos que el algoritmo tiene un costo computacional del orden $\mathcal{O}(n^2)$, y no hay operaciones de orden más alto, a comparación con el método de Newton, que es de orden $\mathcal{O}(n^3)$.
- El algoritmo es robusto, lo cual significa que puede lidiar con errores durante la ejecución en algunos casos, e imputs erróneos, ya que la actualización de H_k es bastante poderosa.

Soluciones

Vemos que los coeficientes resultantes son prácticamente idénticos. Y con una tolerancia de $1e-5$ son idénticos:

```
1  sol_scipy_bfgs.x
```

```
array([-7.87099875e+00, -1.85502456e-02,  6.87997371e-02,  5.42168790e-01,
        3.73328055e-01, -1.08265612e-01,  1.08427557e-02,  4.76587531e-03,
        3.59973058e-03,  1.05642791e-01])
```

```
1  result
```

```
array([-7.87099875e+00, -1.85502512e-02,  6.87997372e-02,  5.42168789e-01,
        3.73328069e-01, -1.08265603e-01,  1.08427521e-02,  4.76587528e-03,
        3.59973049e-03,  1.05642789e-01])
```

```
1  np.allclose(sol_scipy_bfgs.x, result, rtol = 1e-5)
```

```
True
```

Figure 2: Comparamos coeficientes

Soluciones

Obtuvimos la cantidad de iteraciones y el valor de los coeficientes en cada iteración. El algoritmo corrió en un total de 86 iteraciones. Las primeras 10 se ven así, todas bastante variables pues están lejos de converger:

```
1 N_coeficientes.iloc[:,:10]
```

	0	1	2	3	4	5	6	7	8	9
0	-46.5000	-290.732689	-464.306964	-826.301000	-909.831768	-889.266352	-922.489562	-953.455763	-1020.717556	-1159.354542
1	-18.7200	-105.250628	-156.582876	-259.805377	-279.378474	-251.025445	-253.408073	-247.116231	-220.607885	-158.002434
2	-1962.2412	8330.259798	8901.227252	7344.848175	6406.608587	3310.174723	3818.581288	4513.903041	4024.597078	2440.901161
3	-3.4100	137.260987	206.401828	337.859205	374.314668	398.094049	425.724289	463.533021	547.940819	736.100792
4	-1.5300	104.935130	157.821973	258.054627	281.096204	280.440737	298.589527	321.065340	365.246762	461.129760
5	-29.6800	9.170789	-16.570633	-92.998249	-112.262722	-116.644721	-114.775945	-105.985358	-97.241222	-77.386967
6	-23.3100	-135.244457	-220.037174	-396.323832	-440.857358	-455.058134	-472.971806	-491.146205	-545.208836	-665.938598
7	-4887.6756	-6152.923774	-4668.111246	-613.827139	422.902116	845.513549	815.479769	484.996697	398.779702	231.017144
8	-1354.1840	-7092.296601	-10850.171685	-18507.795710	-19833.310679	-17217.063590	-17188.934614	-16201.783555	-13639.337567	-7485.089931
9	-50.5800	-191.882472	-305.868401	-548.050200	-602.478716	-558.910044	-557.587595	-541.157707	-495.564126	-391.880275

Figure 3: Primeras 10 iteraciones

Soluciones

Las últimas 10 iteraciones se ven así, en las que prácticamente ya no hay variación:

```
1 N_coeficientes.iloc[:,-10:]
```

	76	77	78	79	80	81	82	83	84	85
0	-7.870803	-7.871033	-7.870989	-7.870997	-7.870998	-7.870999	-7.870999	-7.870999	-7.870999	-7.870999
1	-0.018643	-0.018504	-0.018560	-0.018551	-0.018550	-0.018550	-0.018550	-0.018550	-0.018550	-0.018550
2	0.068795	0.068800	0.068800	0.068800	0.068800	0.068800	0.068800	0.068800	0.068800	0.068800
3	0.542177	0.542163	0.542169	0.542169	0.542169	0.542169	0.542169	0.542169	0.542169	0.542169
4	0.373275	0.373327	0.373340	0.373327	0.373328	0.373328	0.373328	0.373328	0.373328	0.373328
5	-0.108041	-0.108292	-0.108272	-0.108264	-0.108266	-0.108266	-0.108266	-0.108266	-0.108266	-0.108266
6	0.010900	0.010855	0.010838	0.010843	0.010843	0.010843	0.010843	0.010843	0.010843	0.010843
7	0.004766	0.004766	0.004766	0.004766	0.004766	0.004766	0.004766	0.004766	0.004766	0.004766
8	0.003600	0.003600	0.003600	0.003600	0.003600	0.003600	0.003600	0.003600	0.003600	0.003600
9	0.105666	0.105643	0.105643	0.105643	0.105643	0.105643	0.105643	0.105643	0.105643	0.105643

Figure 4: Últimas 10 iteraciones

Soluciones

Graficamos para ver como los coeficientes van convergiendo:

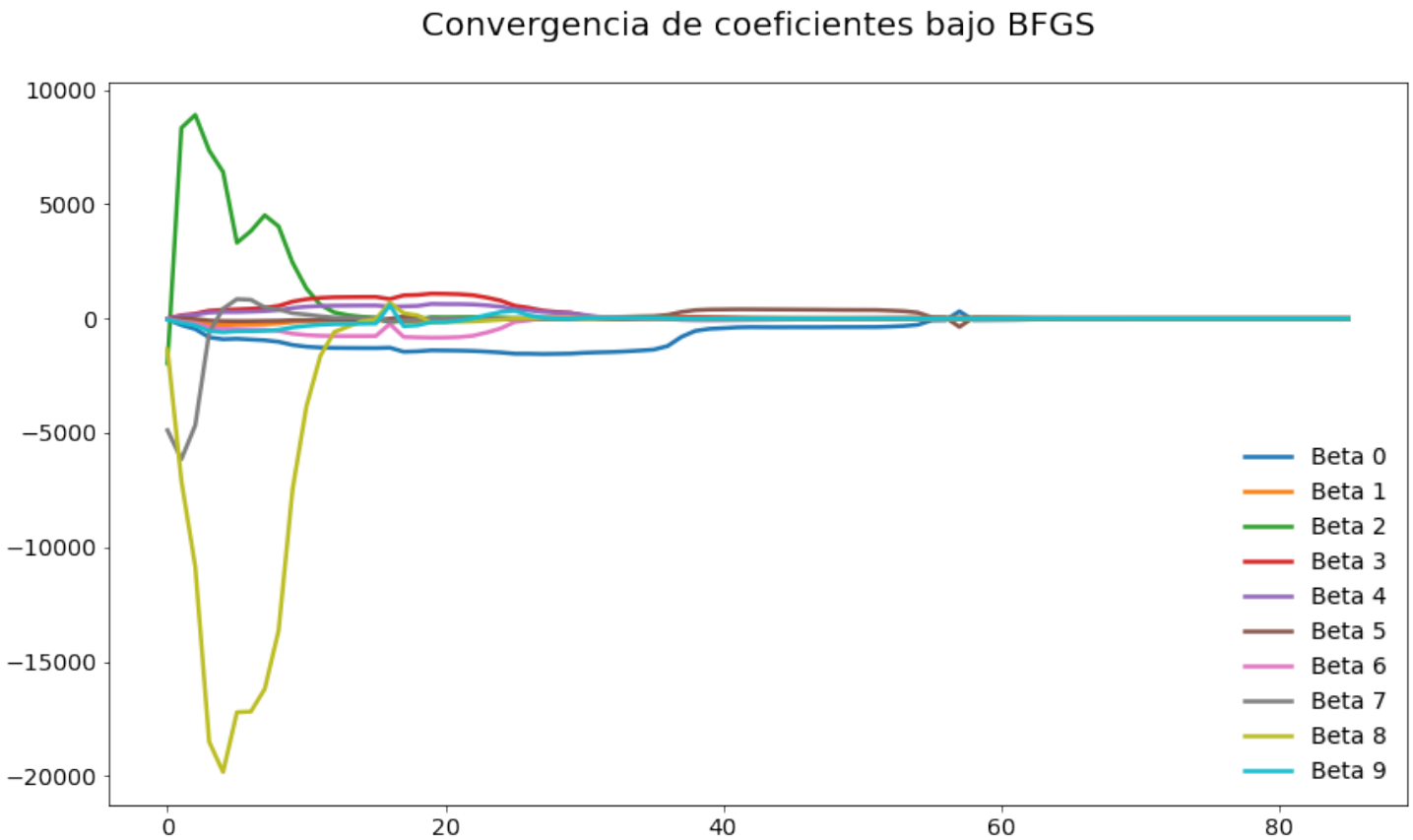
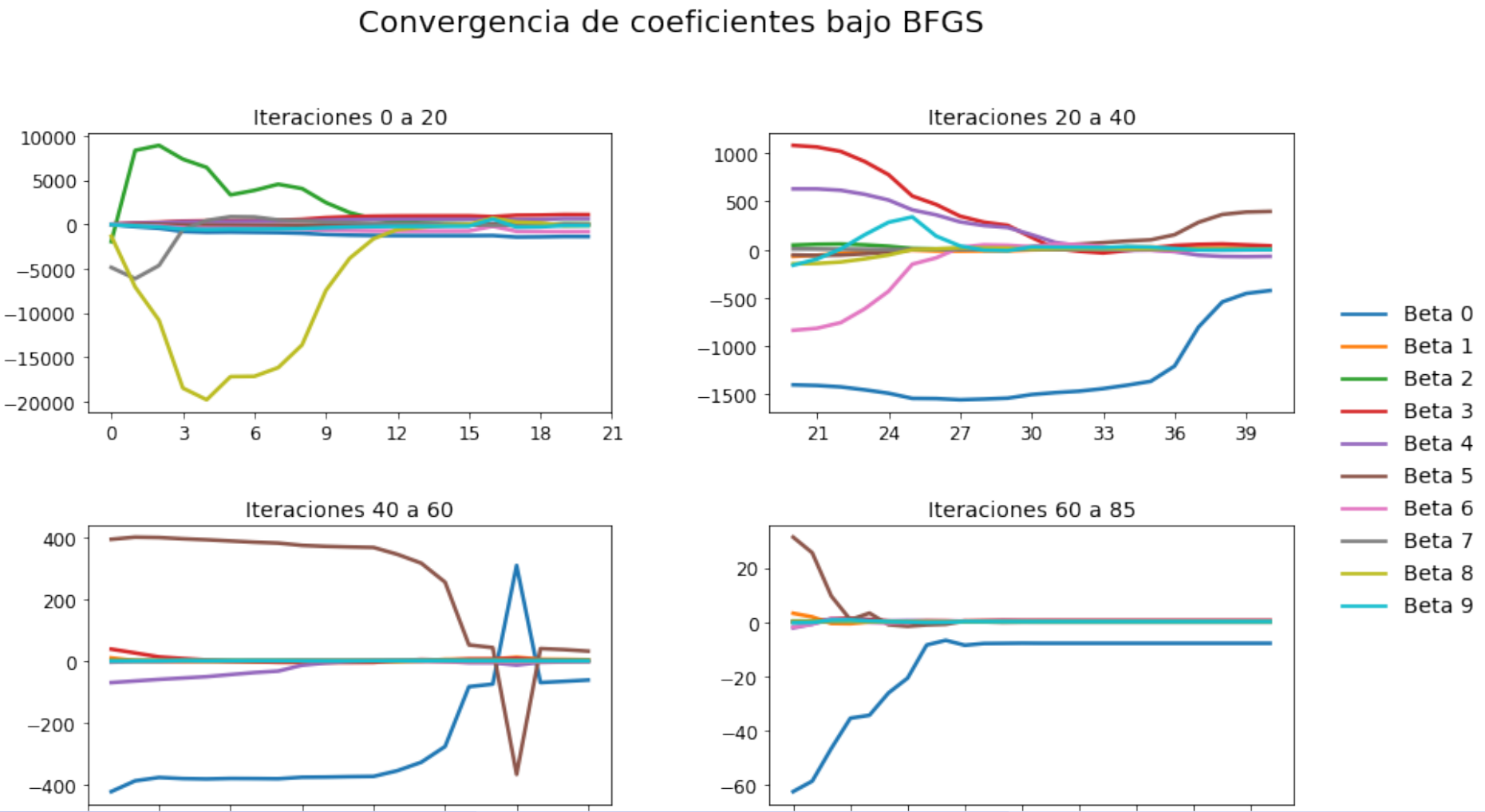


Figure 5: Convergencia de los coeficientes bajo BFGS

Soluciones

Vamos haciendo zoom progresivo para apreciar como se van reduciendo los coeficientes:



Soluciones

Podría además interesarnos como cambia el error relativo entre cada iteración. Graficamos para ver como se comportan los errores relativos de cada coeficiente. Es interesante notar que, aunque haya picos, no necesariamente significa que este mal, sino que fueron puntos de cambio importantes en que los coeficientes se ajustaron para converger:

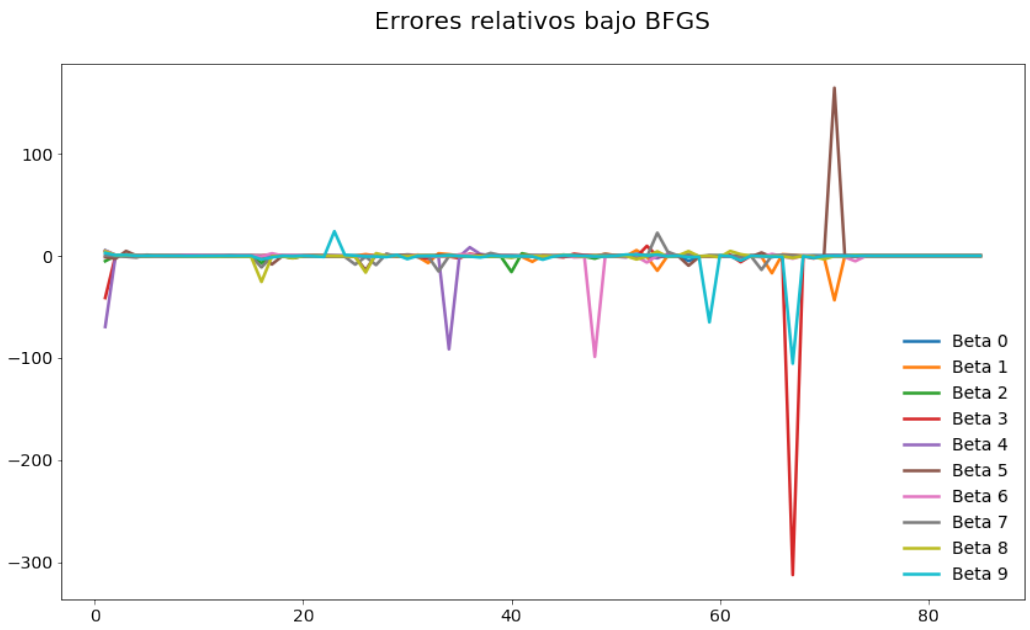


Figure 7: Errores relativos bajo BFGS

Soluciones

Vamos haciendo zoom progresivo:



Predicciones

Realizamos algunas predicciones con los coeficientes calculados para verificar que los resultados sean coherentes.

Los coeficientes son los siguientes:

Variable	Coeficiente
Intercept	-7.870999
gender	-0.018550
age	0.068800
hypertension	0.542169
heart_disease	0.373328
ever_married	-0.108266
Residence_type	0.010843
avg_glucose_level	0.004766
bmi	0.003600
smoking_status	0.105643

Figure 9: Coeficientes

Predicciones : Caso 1

El primer caso es un hombre de 25 años, relativamente saludable (sólo fuma considerablemente).

	X
Intercept	1.00
gender	1.00
age	25.00
hypertension	0.00
heart_disease	0.00
ever_married	0.00
Residence_type	1.00
avg_glucose_level	130.56
bmi	22.10
smoking_status	3.00

Figure 10: Caso 1

Predicciones : Caso 1

Podemos apreciar que en efecto la probabilidad de que este individuo tenga un derrame es bastante baja, lo cual es esperado dado su condición física:

Predicción

```
1 pred = 1/(1 + np.exp(-1 * coefs.T @ new_X))
2 print(f'La probabilidad de tener un derrame es: {np.round(100*pred.iloc[0,0],2)}%')
```

```
La probabilidad de tener un derrame es: 0.58%
```

Figure 11: Predicción 1

Predicciones : Caso 2

El segundo caso es una mujer de 82 años, con varias condiciones desfavorables en su salud:

	X
Intercept	1.00
gender	0.00
age	82.00
hypertension	1.00
heart_disease	1.00
ever_married	0.00
Residence_type	1.00
avg_glucose_level	320.56
bmi	22.10
smoking_status	3.00

Figure 12: Caso 2

Predicciones : Caso 2

En este caso, la probabilidad de que este individuo tenga un derrame es considerablemente alta, lo cual es esperado dado su condición física:

Predicción

```
1 pred = 1/(1 + np.exp(-1 * coefs.T @ new_X))
2 print(f'La probabilidad de tener un derrame es: {np.round(100*pred.iloc[0,0],2)}%')
```

```
La probabilidad de tener un derrame es: 65.05%
```

Figure 13: Predicción 2

Conclusiones

- El método funcionó de manera muy buena. Podemos ver que tenemos más iteraciones que el método implementado por scipy, pero, fuera de eso, la convergencia es a los mismos puntos, y además, tenemos muy alta precisión (del orden de $\approx 10^{-6}$).
- Vemos la potencia del método, ya que este es un problema relativamente complejo, por la forma de las funciones que involucran divisiones por exponentes, que rompen a muchos métodos numéricos.
- Algunas extensiones o proyectos siguientes pueden ser comparar este método contra otros métodos, particularmente BFGS de memoria limitada, o descenso en gradiente.

Referencias

- Nocedal, Jorge, and Stephen Wright. Numerical optimization. Springer Science & Business Media, 2006.
- Simpy
- Stroke-prediction-dataset
- scipy.optimize
- How to minimize with BFGS in Python?
- Python implementation example BFGS

Referencias

- Broyden–Fletcher–Goldfarb–Shanno algorithm
- Line search in gradient and Newton directions
- Wolfe conditions
- Unconstrained Convex Optimization